

Template do Aluno: Mini Projeto "O Arquiteto Decisor"

Aluno: Ana Laura Teodoro Gonçalves | **Matrícula:** 2320597

Link do Repositório GitHub (Scaffolding):
https://github.com/Analauratg05/projeto_arquitetura-.git

CICLO 1: Visão e Requisitos (Fase 1)

Preencher e entregar no AVA ao final do Ciclo 1.

1.1 Resumo do Cenário de Negócio

O HealthSync é uma plataforma de telemedicina que busca integrar consultas online, monitoramento contínuo de pacientes por meio de dispositivos wearables e um sistema de recomendação de tratamentos baseado em Inteligência Artificial. O sistema visa resolver a dificuldade de acesso a cuidados médicos personalizados, permitindo acompanhamento remoto e tomada de decisão clínica mais assertiva. A solução precisa lidar com dados sensíveis de saúde, garantindo segurança, alta disponibilidade e escalabilidade para suportar múltiplos usuários simultâneos, além de integrar diferentes tecnologias como IoT e serviços de IA.

1.2 Atributos de Qualidade (RNFs) Priorizados

[Liste e justifique 5 Requisitos Não Funcionais (RNFs) que são críticos para o sucesso do seu sistema, baseando-se no cenário de negócio. Ex: Desempenho, Segurança, Escalabilidade, Usabilidade, Manutenibilidade, etc.]

- 1 **Segurança:** A proteção dos dados de saúde é essencial, exigindo criptografia, autenticação robusta e controle de acesso, garantindo conformidade com a LGPD.
- 2 **Disponibilidade:** O sistema deve estar continuamente acessível, pois indisponibilidades podem comprometer o atendimento médico e a experiência do usuário.
- 3 **Escalabilidade:** A arquitetura deve suportar crescimento de usuários e picos de demanda, especialmente em consultas simultâneas e processamento de dados.
- 4 **Integridade de Dados:** Os dados clínicos devem permanecer corretos, consistentes e protegidos contra alterações indevidas.

- 5 **Performance:** As recomendações de tratamento devem ser geradas rapidamente, com tempo de resposta adequado (por exemplo, até 2 segundos para a maioria das requisições).

1.3 Diagrama de Contexto (C4 Nível 1)

https://github.com/Analauratg05/projeto_arquitetura-/blob/main/diagrams/diagrama_swimlane_limpo.png

1.4 Classificação da Estratégia

- **Classificação:** Balanceada
 - **Justificativa:** A estratégia balanceada foi adotada para equilibrar inovação e controle de riscos, utilizando tecnologias modernas como Inteligência Artificial e IoT sem renunciar a práticas consolidadas. Essa abordagem reduz incertezas técnicas ao apoiar-se em arquiteturas já maduras, como microsserviços e computação em nuvem. Ao mesmo tempo, permite evolução contínua do sistema, característica essencial em software. Conforme Pressman, o software possui natureza evolutiva e adaptativa, exigindo decisões que considerem mudanças e complexidade.
-

● CICLO 2: Blueprint e Decisões (Fase 2)

2.0 Saneamento da Fase 1

Durante a revisão arquitetural foi identificado o risco de acoplamento entre a camada de negócio e a infraestrutura no modelo N-Tier inicialmente considerado. Conforme a Regra da Dependência (Martin, 2017), o domínio não pode depender de frameworks, banco ou detalhes técnicos. Isso geraria:

- Alto acoplamento
- Baixa testabilidade
- Dívida técnica precoce (Pressman, 2011)

Refatoração aplicada

O sistema HealthSync passa a adotar: Arquitetura Hexagonal (Ports and Adapters) dentro de um ecossistema de microsserviços.

2.1 Diagrama de Containers (C4 Nível 2)

https://github.com/Analauratg05/projeto_arquitetura-/blob/main/diagrams/healthsync_c4_container_v2.png

2.2 Estilo Arquitetural Escolhido

Arquitetura: Microserviços + Hexagonal + Event-Driven

A arquitetura combina três estilos para atender os RNFs críticos.

c

Trade-off 1 — Escalabilidade vs Complexidade

Pró:

Microserviços permitem escalar apenas partes críticas (ex: IA e teleconsulta).

Contra:

Aumenta complexidade operacional e DevOps.

Impacto RNF: Escalabilidade ↑ Disponibilidade ↑

Trade-off 2 — Desacoplamento vs Latência

Pró:

Mensageria desacopla serviços IoT e IA.

Contra:

Processamento assíncrono pode gerar latência eventual.

Impacto RNF: Confiabilidade ↑ Performance ↓ (leve)

Trade-off 3 — Segurança vs Custo

Pró:

Separação de serviços permite:

- isolamento de dados sensíveis
- políticas LGPD

Contra:

Mais serviços → maior custo cloud.

Impacto RNF: Segurança ↑ Custo ↑

2.3 Architecture Decision Record (ADR) Principal

[Escolha uma decisão arquitetural crucial que você tomou nesta fase (ex: Escolha da Tecnologia de Persistência, Estratégia de Comunicação entre Microserviços) e documente-a como um ADR. **Recomendado: Criar o arquivo .md do ADR em /adrs no GitHub e referenciar o link aqui.**]

- **Título:** Adoção de Arquitetura Hexagonal nos Microserviços do HealthSync
- **Status:** Aceito
- **Contexto:**
 - O modelo N-Tier tradicional gerava risco de:

Domínio dependente de banco/framework

Dificuldade de testes automatizados

Alto acoplamento com infraestrutura

Risco elevado de dívida técnica

Além disso, o sistema precisa integrar:

IoT

IA

APIs externas

Banco de dados múltiplos

Isso exige forte isolamento do domínio.

- **Decisão:** Adotar **Arquitetura Hexagonal (Ports and Adapters)** dentro de cada microserviço.
- Estrutura interna de cada serviço:
- Camadas internas:

Domínio (regras médicas)

Casos de uso (Application)

Adaptadores externos:

Banco de dados

APIs externas

Mensageria

Framework web

Comunicação:

REST + eventos assíncronos

- **Justificativa:** A adoção da Arquitetura Hexagonal foi motivada pela necessidade de proteger o núcleo. A Arquitetura Hexagonal foi adotada para evitar o acoplamento entre o domínio e a infraestrutura, problema comum no modelo N-Tier. No HealthSync, onde há regras clínicas e dados sensíveis, depender de banco, frameworks ou APIs comprometeria a evolução, a testabilidade e a manutenção do sistema. Com essa abordagem, o domínio permanece isolado e independente, enquanto integrações externas são tratadas como adaptadores substituíveis. Isso permite trocar tecnologias sem impactar as regras de negócio e facilita a realização de testes sem necessidade de infraestrutura. Além disso, a decisão reduz o risco de

dívida técnica, evitando que escolhas tecnológicas influenciem o núcleo do sistema. Assim, a Arquitetura Hexagonal garante baixo acoplamento, maior flexibilidade e melhor aderência aos requisitos de segurança e manutenibilidade.

CICLO 3: Cloud e Resiliência (Fase 3)

Preencher e entregar no AVA ao final do Ciclo 3.

3.1 Estratégia de Cloud e Implantação

[Descreva como sua arquitetura será implantada em um ambiente de nuvem (ex: AWS, Azure, GCP). Qual o modelo de serviço (IaaS, PaaS, Serverless)? Como o sistema será escalado e monitorado?]

O sistema HealthSync será implantado na Amazon Web Services utilizando uma estratégia híbrida baseada em microsserviços containerizados e serviços gerenciados em nuvem.

Modelo de Serviço

A arquitetura utilizará principalmente:

PaaS (Platform as a Service) para banco de dados, mensageria e monitoramento.

IaaS + Containers para execução dos microsserviços.

Serverless para tarefas específicas assíncronas e processamento de eventos.

Componentes da Implantação

O componente utilizará esses:

Microsserviços executados em containers Docker orquestrados pelo Kubernetes via Amazon EKS.

API Gateway para centralização das requisições externas.

Banco PostgreSQL gerenciado utilizando Amazon RDS.

Comunicação assíncrona entre serviços utilizando mensageria com filas e eventos.

Serviços de IA executados separadamente para permitir escalabilidade independente.

Dados de dispositivos IoT recebidos via endpoints seguros e processados em pipelines assíncronos.

Escalabilidade

A escalabilidade será horizontal e automática:

Kubernetes realizará auto-scaling dos pods conforme uso de CPU e memória.

Serviços críticos como teleconsulta e IA poderão escalar independentemente.

Load balancers distribuirão requisições entre múltiplas instâncias.

Processamento assíncrono reduzirá gargalos em horários de pico.

Monitoramento

O sistema utilizará:

Logs centralizados.

Métricas de infraestrutura e aplicação.

Monitoramento contínuo de disponibilidade.

Rastreamento distribuído entre microsserviços.

Alertas automáticos para falhas críticas.

3.2 Análise de Fragilidade e Mitigação

- **Ponto Frágil:** [Identifique a maior fraqueza ou risco da sua arquitetura em um cenário de produção (ex: falha de um serviço crítico, pico de tráfego inesperado, vulnerabilidade de segurança).]

O principal risco arquitetural está relacionado à comunicação distribuída entre microsserviços, principalmente nos serviços críticos de:

- teleconsulta,
- processamento de IA,
- e integração com dispositivos IoT.

Falhas em um serviço podem gerar efeito cascata, aumentando latência, indisponibilidade ou inconsistência de dados clínicos, outro risco importante é o aumento repentino de acessos simultâneos durante períodos de alta demanda.

- **Mitigação:** [Como você minimizaria esse risco? Quais estratégias de resiliência (ex: circuit breaker, retry, fallback, multi-região) seriam aplicadas?]

Para reduzir esses riscos, serão aplicadas estratégias de resiliência:

Circuit Breaker

Evita propagação de falhas entre serviços, interrompendo chamadas para serviços indisponíveis temporariamente.

Retry com Backoff

Requisições temporariamente falhas serão reenviadas automaticamente com tempo progressivo de espera.

Fallback

Caso serviços secundários falhem, o sistema poderá fornecer respostas alternativas ou operar parcialmente.

Auto-scaling

Os microsserviços escalam automaticamente conforme aumento de carga.

Mensageria Assíncrona

Eventos críticos serão desacoplados usando filas para evitar perda de dados e reduzir dependência síncrona.

Replicação e Backup

O banco de dados possuirá replicação e backups automáticos para recuperação de desastres.

Segurança e LGPD

Serão aplicadas:

- criptografia em trânsito e em repouso,
- autenticação robusta,
- segregação de dados,
- e controle de acesso baseado em papéis.

3.3 Parecer Técnico Final

[Resumo executivo (até 10 linhas) defendendo por que sua arquitetura é a melhor escolha para o cliente, considerando os requisitos de negócio, os RNFs e a estratégia de implantação. Qual o valor agregado da sua solução?]

A arquitetura proposta para o HealthSync combina microsserviços, Arquitetura Hexagonal e processamento orientado a eventos para atender os requisitos críticos do sistema de telemedicina. A solução garante escalabilidade, segurança e alta disponibilidade, fundamentais para aplicações médicas que lidam com dados sensíveis e atendimento contínuo. A utilização de

cloud computing na AWS permite crescimento sob demanda, monitoramento avançado e resiliência operacional. Além disso, o desacoplamento entre domínio e infraestrutura reduz dívida técnica e facilita evolução futura do sistema. Embora exista aumento de complexidade operacional, os benefícios em flexibilidade, confiabilidade e capacidade de expansão justificam a escolha arquitetural. Dessa forma, o HealthSync obtém uma plataforma moderna, segura e preparada para suportar crescimento contínuo e integração com tecnologias de IA e IoT.

📌 BÔNUS: Evolução Arquitetural (Gamificação)

Opcional - Preencher apenas se houver melhoria estrutural documentada no GitHub.

[Descreva brevemente a melhoria arquitetural que você implementou no seu repositório GitHub para fins de bônus de nota. Referencie a nova ADR e o diagrama atualizado (se houver) no GitHub. Ex: Migração de um componente para Serverless, implementação de um padrão de resiliência, otimização de custo em cloud. **Lembre-se: a melhoria deve ser arquitetural, não apenas textual.**]

SURPREENDA-ME! 😊

📖 Referências Bibliográficas

- **Pressman, R. S.** (2021). *Engenharia de Software: Uma Abordagem Profissional*. McGraw Hill. (Capítulos selecionados conforme o tema da aula).
- **Richards, M., & Ford, N.** (2020). *Fundamentals of Software Architecture: An Engineering Approach*. O'Reilly Media.
- **C4 Model for Software Architecture.** (s.d.). Disponível em: <https://c4model.com/>
- **Architecture Decision Records (ADRs).** (s.d.). Disponível em: <https://adr.github.io/>